Published: 09 March 2015
Accepted: 01 January 2015
Revised: 01 January 2015
Received: 01 March 2014

RESEARCH-ARTICLE

Metrics and Algorithms for Routing Questions to User Communities

ADITYA PAL, IBM Research - Almaden, San Jose, CA, United States

Metrics and Algorithms for Routing Questions to User Communities

ADITYA PAL, IBM Research

An online community consists of a group of users who share a common interest, background, or experience, and their collective goal is to contribute toward the welfare of the community members. Several websites allow their users to create and manage niche communities, such as Yahoo! Groups, Facebook Groups, Google+ Circles, and WebMD Forums. These community services also exist within enterprises, such as IBM Connections. Question answering within these communities enables their members to exchange knowledge and information with other community members. However, the onus of finding the right community for question asking lies with an individual user. The overwhelming number of communities necessitates the need for a good question routing strategy so that new questions get routed to an appropriately focused community and thus get resolved in a reasonable time frame.

In this article, we consider the novel problem of routing a question to the right community and propose a framework for selecting and ranking the relevant communities for a question. We propose several novel features for modeling the three main entities of the system: questions, users, and communities. We propose features such as language attributes, inclination to respond, user familiarity, and difficulty of a question; based on these features, we propose similarity metrics between the routed question and the system entities. We introduce a CUTOFF-AGGREGATION (*CA*) algorithm that aggregates the entity similarity within a community to compute that community's relevance. We introduce two *k*-nearest-neighbor (*knn*) algorithms that are a natural instantiation of the *CA* algorithm, which are computationally efficient and evaluate several ranking algorithms over the aggregate similarity scores computed by the two *knn* algorithms. We propose clustering techniques to speed up our recommendation framework and show how pipelining can improve the model performance. We demonstrate the effectiveness of our framework on two large real-world datasets.

Categories and Subject Descriptors: H.1.2 [**Information Systems**]: User/Machine Systems—*Human information processing*; H.3.3 [**Information Storage and Retrieval**]: Information Search and Retrieval—*Retrieval models*

General Terms: Experimentation, Human Factors, Algorithms

Additional Key Words and Phrases: Question answering, community question routing, group recommendation

ACM Reference Format:
Aditya Pal. 2015. Metrics and algorithms for routing questions to user communities. ACM Trans. Inf. Syst. 33, 3, Article 14 (March 2015), 29 pages.
DOI: http://dx.doi.org/10.1145/2724706

1. INTRODUCTION

An online community is a group of users who interact with one another through Internet technologies [Preece and Krichmar 2005]. The members of the community generally share a common interest, background, or experience, and their collective goal is to contribute toward the welfare of the community members [Sproull and Arriaga 2007].

This work extends our prior published work [Pal et al. 2013].

Author's address: A. Pal, IBM Almaden Research Center, 650 Harry Road, San Jose, CA; email: apal@us.ibm.com.

DOI: http://dx.doi.org/10.1145/2724706

Many online services provide mechanisms to create niche communities, for example, Yahoo! Groups,[1] Facebook Groups,[2] Google+ Circles,[3] and WebMD Forums.[4] Similar community services also exist within enterprises and cater to their employees, such as IBM Connections.[5] Similar to the growth of online question-answering services, such as Yahoo Answers[6] and Stack Overflow,[7] which attracts millions of users daily, the popularity of question answering within community services is on the rise. As an example, WebMD has a dedicated online community portal that allows users to join communities, create new communities, and post questions and discussions within these communities. Similarly, IBM has a dedicated internal online community portal called IBM Connections. There are more than 150,000 communities in IBM Connections, and question answering plays an essential role in enabling information flow and collaboration among the community members.

However, the onus of finding the right community for question asking lies with the individual user. Question askers risk their reputation while asking because they can be easily rebuked for asking in the wrong community. Moreover, the overwhelming number of communities makes their choice much harder. As a result, an automated *question routing* strategy needs to be devised, so that a new question gets routed to the appropriately focused community and thus is resolved in a reasonable time frame. This would alleviate the burden of finding the right community for the question asker and eliminates the risk of earning community members' disapproval. As a concrete example, there are more than 15 communities on WebMD that relate to *parenting*. These communities range from *cloth diaper* to *parenting: 2-year-olds* to *autism support group*. For uninitiated question askers, it would require a considerable effort to sift through the large volumes of data in order to identify the most relevant community for asking their question.

In the traditional setting, question routing refers to finding the best answerers for a question, but in our setting, it refers to finding the right communities to provide an answer. While question routing to the appropriate answerer is an important concept and prior studies [Guo et al. 2008; Zhou et al. 2009; Li et al. 2010, 2011; Chang and Pal 2013] show that it enhances the user experience, one of the largely ignored areas is question routing to the appropriate communities instead of individuals. The notion of routing to a community is useful as it increases the likelihood of the question being answered. This is because community is a collection of people, and hence the chances of more eyes looking at the question are higher. Further, it prevents clogging the bandwidth of the preferred experts that are otherwise top targets for routing algorithms and gives a fair chance to other users as well. Another important factor is that the collective knowledge of the community or the *wisdom of the crowd* supersedes that of an individual. Additionally, there are scenarios where a user is looking for a focused community and not an individual, such as a support group where users can share their problems and comfort one another (WebMD is replete with such use cases).

One of the main challenges for routing questions to communities is the task of modeling communities. There is extensive research on modeling user behavior, but how do we aggregate that to capture community behavior? Simple aggregation techniques, such as *min*, *mean*, *max*, and *mean*, do not work because they fail to take into account

[1]https://groups.yahoo.com/neo.
[2]https://www.facebook.com/about/groups.
[3]https://www.google.com/+/learnmore/circles/.
[4]http://exchanges.webmd.com.
[5]https://www.ibm.com/developerworks/lotus/products/connections.
[6]http://www.answers.yahoo.com.
[7]http://www.stackoverflow.com.

the fact that a community consists of a wide variety of people with asymmetric roles and participation; the majority are knowledge seekers or lurkers, and a small proportion of the people are knowledge creators or volunteers. A second challenge is how do we capture community norms and practices? Adherence of the question to community norms can be critical while routing. To see this, consider two communities: *homework discussion* and *programming experts*. Ideally, a programming question would get better responses in the latter community; however, a homework question on programming would be banned and the question asker would be admonished there, whereas the same question would be welcomed in the former community. Hence, community norms can be an important attribute besides interests of individual users.

In order to address these challenges, we build upon the prior state of the art in topic modeling and expertise identification and propose some novel metrics, such as language analysis of questions, difficulty of questions, inclination to respond, and community characteristics such as community guidelines. We propose similarity metrics between the routed question and the system entities (questions, users, communities). The similarity metrics between the routed question and the existing questions (users) are individual similarity metrics. In order to aggregate individual similarity metrics to compute community similarity, we propose a CUTOFF-AGGREGATION (*CA*) algorithm. We show how the *CA* algorithm can model simple aggregation techniques such as *min*, *max*, *mean*, and *median*. We then propose two *k*-nearest-neighbor (*knn*) algorithms, which are a natural instantiation of the *CA* algorithm for aggregating the individual similarity metrics to get a community score. The two algorithms are carefully constructed so that the aggregation can be computed quite efficiently (linear time in input). We evaluate several ranking models that combine the community scores aggregated through the *knn* algorithm to produce a ranked list of communities. We also propose a data-partitioning scheme to speed up the recommendations. We also propose a pipelining scheme to improve the model performance even further. Our proposed framework can run in near real time and is thus appropriate for large-scale community datasets. We conduct experiments over two large real-world datasets in order to show the effectiveness of our recommendation framework.

1.1. Our Contributions

The main contributions of this article are as follows:

(1) We introduce the novel problem of routing questions to user communities. Traditional algorithms of routing questions to the best answerers do not address this problem. We propose a set of metrics to model the features of communities, their members, and the questions asked by them. Based on these features, we propose several novel similarity metrics, such as relevance of the question w.r.t community guidelines, familiarity of the question asker with the community members, value of a question, and difficulty level of a question.
(2) We propose a CUTOFF-AGGREGATION algorithm to aggregate the individual similarity metrics to compute the community scores. We show that the *CA* algorithm can easily model simple aggregation techniques, such as, *min*, *max*, *mean*, and *median*. We then propose two *knn* algorithms, which are a natural instantiation of the *CA* algorithm. We empirically show that the two *knn* algorithms are significantly more effective than the *min*, *max*, *mean*, and *median* aggregation strategy for the routing problem.
(3) We evaluate several ranking algorithms and show their effectiveness over two real-world datasets. We also propose a dataset-partitioning scheme that leads to a more than 50% reduction in time without much drop in performance (7%–9%). We propose a pipelining strategy to improve the performance of our framework.

(4) We evaluate the effectiveness of our overall framework over two real-world datasets and analyze the features that are most predictive for the community question routing problem. The overall framework is constructed with the goal of providing recommendations in near real time and is easy to implement over a MapReduce-based framework [Dean and Ghemawat 2008].

2. RELATED WORK

We divide the related work into four parts: (1) expertise identification, (2) question routing, (3) question classification, and (4) Group recommendation.

2.1. Expertise Identification

Feature-based and graph-based approaches for finding experts in question-answering communities (CQAs) have been applied by several researchers. Bouguessa et al. [2008] proposed a feature-based model to identify experts in CQA. They considered the number of best answers given by a user as an indicator of authority of that person. In their approach, they modeled the authority scores of users as a mixture of gamma distribution and used Bayesian Information Criteria (BIC) to estimate the appropriate number of mixtures and computed the parameters of each mixture component using the Expectation Maximization (EM) algorithm. The suggested benefit of their approach over graph algorithms is that their approach automatically surfaces the number of experts in the community, whereas for graph algorithms, the number of experts is considered known beforehand (which might not be desirable).

Zhang et al. [2007] proposed ExpertiseRank, a slight variant of PageRank, which considers not only how many people one helped but also whom they helped. They also proposed a feature-based measure called Z_{score}, which is measured based on the number of answers (a) and number of questions (q) as $Z_{score} = \frac{a-q}{\sqrt{a+q}}$. A user with a higher Z_{score} is more likely to be an expert than a user with a lower Z_{score}. This implies that experts answer a lot of questions and ask very few questions (often zero). Their results indicate that for the Java developer forum, simple measures like Z_{score} perform better than more complex graph-based algorithms like PageRank, ExpertiseRank, and HITS. The authors concluded that the network's structural characteristics (i.e., how users are interconnected) matters for the effectiveness of expertise-ranking algorithms.

Jurczyk and Agichtein [2007] performed link analysis on a dataset from Yahoo! Answers by using a slight adaptation of the HITS algorithm. Their results indicate that the HITS algorithm outperformed simple graph measures (such as in-degree-based algorithms). Pal et al. [2012] proposed a question selection bias-based framework to identify experts in CQA. They showed that their model does not need a lot of data, which can be useful during cold start.

2.2. Question Routing

The problem of question routing in question-answering services has been recently explored by several research works [Guo et al. 2008; Zhou et al. 2009; Li et al. 2011, 2010; Lai and Kao 2012; Chang and Pal 2013].

Guo et al. [2008] proposed a technique for recommending questions to users based on their interests.They proposed a generative model that could tap users' participation in CQA effectively by discovering their topic inclination and expertise and then recommending appropriate questions. Zhou et al. [2009] employ the content and structure of question/answer threads to find the top-k potential experts to answer a given question. First, they capture the expertise of the users through three models: (1) profile based, (2) content based, and (3) thread based. The expertise indicates the probability of a user's proficiency in answering a given question. Then users are ranked based

on this structural relationship with other users. Their evaluation on the *TripAdvisor* dataset reveals that expertise computation and ranking of users on expertise scores is an effective way to recommend potential answerers to a question.

Li et al. [2010] present a routing approach in a programming language forum. They use semantic information from user posts along with the post-reply relation to find the experts. They extend the prior work by taking into account the programming language constructs to understand the query and improve answerer search. Li et al. [2011] exploit the hierarchical category information present with questions in `Yahoo! Answers`. Once a question gets categorized, the task is to find the users who are most likely to answer that question given its category. Their experiments over the `Yahoo! Answers` dataset show that taking categories into account improves the routing performance.

Recently, Chang and Pal [2013] proposed a question routing scheme that aims at selecting the best subset (of a fixed size k) of users, who would collaboratively answer a question. The authors hypothesize that question answering is more interactive and a single answer might not suffice. As a result, several users who have different specializations (answering, voting, commenting) are required to ensure that the answer has long-lasting value. They use the StackOverflow dataset to provide empirical evidence to their theory and propose a recommendation model for selecting the best subset of compatible users for answering a question.

2.3. Question Classification

Another related problem is question classification, which aims at putting the question into several semantic categories [Zhang and Lee 2003]. This is done by building a category profile based on the prior categorized questions. There are approaches to find similar questions (based on topics) [Cao et al. 2008] that can be leveraged for question classification. However, typically, these categories are orthogonal and they do not share topical similarities, whereas communities do not follow such constraints. In fact, in most cases, communities compete with one another in the topic space. They might be distinguishable from one another based on membership demographics (age, role, location), whereas these aspects are not critical for question categorization. Additionally, the notion of user membership and their intracommunity dynamics is not present during question categorization but is pivotal for routing.

2.4. Group Recommendation

A closely related problem to routing to communities is group recommendation [O'Connor et al. 2001; Amer-Yahia et al. 2009; Gartrell et al. 2010]. Group recommendation is a task of recommending items (such as games, movies, and restaurants) to a group of users instead of a single user. However, the key difference is that for group recommendation, the recommended item must appeal to all group members. As a result, *min*-, *max*-, or *mean*-based aggregation works well in practice for group recommendation [O'Connor et al. 2001]. Recently, approaches have been proposed that blend content-based methods with collaborative filtering to solve the group recommendation problem [Ye et al. 2012] or through latent feature models based on information matching [Gorla et al. 2013]. One key difference of the our problem from the group recommendation is that the same item can be recommended to several groups simultaneously (e.g., an Oscar-winning movie might appeal to several groups), but a question should ideally be asked in only one community (otherwise, the system managers might consider it as a spam behavior and ban the user). Additionally, for the community routing problem, aspects of the community also come into play, which need not be very effective for group recommendation.

Prior work in expertise identification and question routing focused on finding the relevant individuals for answering a question. Our work differs from most prior work

on question routing, as it focuses on finding the right community for a question. For our problem, the community guidelines, norms, and practices as well as the collective interest of its members must be considered while routing. This problem has been explored to some extent in our prior work [Pal et al. 2013], and we extend our prior work in following ways:

(1) We propose the general community question routing problem and show how we can relax the problem to make it tractable.
(2) We explore several new features such as difficulty of question, part-of-speech tagging, number of views, and question length. These features were not explored by our prior work. We also extend the similarity metrics to incorporate these new features.
(3) We propose a novel algorithm called the CUTOFF-AGGREGATION (*CA*) algorithm for aggregation of the similarity metrics and show how simple aggregation strategies such as *min*, *max*, *mean*, and *median* can be modeled through the *CA* algorithm. We also show how the prior proposed *knn* algorithms are instantiation of the *CA* algorithm. We provide detailed proofs where necessary.
(4) We propose a pipelining algorithm that improves the performance beyond our prior work. We also propose a strategy for data partitioning, which leads to a significant reduction in the computation time of the recommendation, with only a slight decrease in performance.
(5) We extend our analysis to a freely available dataset. Our prior work was done on IBM's internal dataset. We now include experiments on a WebMD dataset. We show how the additions proposed in this article, such as new features and modified similarity metrics, improve the model performance further. We run several new experiments to show the effectiveness of our routing framework.

3. COMMUNITY QUESTION ROUTING PROBLEM

Let $\mathcal{U} = \{u_1, u_2, \ldots, u_n\}$ be the set of n users and $\mathcal{C} = \{c_1, c_2, \ldots, c_m\}$ be the set of m communities, such that $u_i \mapsto c_j$ indicates that the user u_i is a member of the community c_j. A user can be member of multiple communities simultaneously.

Problem 1 (*Community Question Routing*). Given a question q asked by a user u, find a set of communities $C_k^\star$ of maximum size k, such that the following holds:

$$C_k^\star = \arg\max_{C \subseteq \mathcal{C}} \left\{ \mathcal{V}alue_{q,u}(C) - \lambda |C| \right\} \tag{1}$$

subject to

$$1 \leq |C| \leq k$$
$$u \mapsto c, \forall c \in C,$$

where $\mathcal{V}alue_{q,u}(C) \geq 0$ is an estimate of the quality of the answers generated by the members of the communities C to question q. The regularization parameter λ controls the number of selected communities.

The $\mathcal{V}alue$ function can be an arbitrary function; however, due to the nature of our domain, we note that it must be a submodular monotone function (i.e., it must satisfy the following two properties for all subsets of communities $A \subseteq B$):

$$\mathcal{V}alue(A \cup x) - \mathcal{V}alue(A) \geq \mathcal{V}alue(B \cup x) - \mathcal{V}alue(B) \quad \text{(submodularity)} \tag{2}$$

$$\mathcal{V}alue(A) \leq \mathcal{V}alue(B) \quad \text{(monotonicity)}. \tag{3}$$

The intuitive justification for these two properties are as follows: routing to additional communities should yield diminishing returns (submodularity) but no loss (monotone).

Note that we have dropped the subscripts q, u for notational simplicity. Despite *Value* being a submodular monotone, the general *CQR* problem is NP-hard.

LEMMA 3.1. *CQR problem is NP-hard.*

PROOF. Consider the following definitions:

$$\mathcal{V}alue(x) \;=\; \text{number of experts in community } x \tag{4}$$

$$\mathcal{V}alue(C) \;=\; \mathcal{V}alue\left(\bigcup_{x\in C} x\right)(\textit{number of distinct experts in communities } C) \tag{5}$$

$$U \;=\; u : u \to c, \forall c \in \mathcal{C}(\textit{universe of all experts}). \tag{6}$$

It is easy to verify that $\mathcal{V}alue$ is a submodular monotone function. Now, if we set $\lambda = 0$ and $k = |\mathcal{C}|$, we recover the maximum coverage problem—select at most k subsets such that the maximum number of the elements (as measured by $\mathcal{V}alue$) in the universe U are covered. So the maximum coverage problem reduces to the CQR problem through the previous polynomial-time substitutions. Since the maximum coverage problem is NP-hard, the CQR problem is NP-hard. □

The time complexity of the CQR problem is $O(|U| \cdot 2^{|\mathcal{C}|})$, as the number of subsets that need to be evaluated is exponential in the input: number of communities.

3.1. Relaxed Community Question Routing Problem

We relax the CQR problem in order to make it tractable. We make an assumption that the answers provided in a community are independent of the answers provided in other communities. This might not be true in general due to membership overlap between the communities. This assumption leads to the following decomposition of the value function:

$$\mathcal{V}alue(C) = \sum_{c\in C} \mathcal{V}alue(c). \tag{7}$$

Clearly, since $\mathcal{V}alue(C) \geq 0$, the linearity of the value function obeys the submodular monotone property. Based on the linear decomposition of the value function, the problem can be simplified to finding the top-k communities for routing.

Problem 2 (*Relaxed Community Question Routing*). Given a question q asked by a user u, find a community $C_k^\star$, such that the following holds:

$$C_k^\star = \arg \max_{C\subseteq\mathcal{C}, |C|\leq k} \left\{ \sum_{c\in C} \mathcal{V}alue_{q,u}(c) - \lambda|C| \right\}. \tag{8}$$

Problem 2 can be solved through a greedy algorithm. First, sort the communities in decreasing order of their value, and then select one community at a time, picking from the top, until the objective value starts to decrease or k communities get selected. This greedy approach provides an optimal solution to the RCQR problem and it can be shown using Nemhauser et al. [1978] to be $1 - 1/e$ close to the optimal solution of the CQR problem.

A key point to note here is that a question asker must be a member of the community. This constraint could lead to a search within the set of subscribed communities, which might not work nicely for a new user. Hence, we relax this constraint and allow a search among all the communities. Once a relevant community is found, users can be recommended to join that community and then ask their questions.

Table I. Question Features (note that some of the basic features $QB1, \ldots, QB5$ are not present for the routed question)

Type	Id	Feature
Basic Features	$QB1$	Number of views
	$QB2$	Number of unique answerers
	$QB3$	Did asker answer the question
	$QB4$	Votes on the answers
	$QB5$	Time lapsed b/w question and the first answer
	$QB6$	Length of the question
Content Features	$QC1$	Word vector (normalized using Tf-Idf)
	$QC2$	Topic distribution (LDA, LSA, etc.)
	$QC3$	Language analysis (e.g. nouns, pronouns, sentiment, politeness, openness)
	$QC4$	Part of speech tagging
	$QC5$	Difficulty level of the question
	$QC6$	Question norms (etc, typos, greetings, curse words, length, negative sentiment)

4. METRICS FOR COMMUNITY QUESTION ROUTING

There are three types of entities that need to be modeled for the community question routing problem. The first entity is the *question,* which includes the routed question and the existing questions asked in the communities. The second entity is the *user,* which includes the question asker and the community members. Finally, the third entity is the *community,* where the interaction between the users occurs. In this section, we present details of features that are computed for these three entities.

4.1. Question Features

A question typically consists of a summary title and an optional description. We combine these two fields together to create a question document (QD). Based on QD and question metadata, we extract several different features as listed in Table I. Most of these features are straightforward to compute and self-explanatory.

We compute questions' topic distribution ($QC2$) through the Latent Dirichlet Allocation (LDA) [Blei et al. 2001] algorithm, which is currently the most popular topic modeling algorithm. To run LDA, we first combine all QDs within a community to create a community document CD. Then, LDA is run over CDs, and once it estimates the topic-word proportions, we infer the topic distribution of an individual question. We run the topic model over CDs instead of QDs directly because it leads to a more reliable estimation of topics due to two main facts: (1) there are fewer communities (hundreds to thousands) in comparison to questions (tens or hundreds of thousands), and (2) CDs are wordy ($\geq$7,000 words), whereas QDs are terse ($\sim$100 words). These factors make CDs more robust to noise and thus a more reliable topic estimation.

We carried out a language analysis ($QC3$) of the question text using the Linguistic Inquiry and Word Count (LIWC) tool.[8] LIWC provides scores on 80 to 90 features for the input text. The most prominent of those features are (1) sentiment, (2) openness, (3) usage of nouns and pronouns, (4) usage of greetings, and (5) usage of special characters (e.g., @, #, ?, !, etc.). We also compute several different norm features ($QC6$) that can be used to identify whether a question adheres to the guidelines of a community or not. These features are (1) usage of bad words, (2) length of the question, (3) usage of negative sentiments, and (4) typos. In order to compute the typos and spelling mistakes, we use a dictionary (English words + Technical jargon) and the Jaro-Winkler

[8]http://www.liwc.net.

[Jaro 1989] similarity measure. The norms features enable us to test whether a question adheres to the community norms (e.g., censorship of some communities toward bad words, bad readability of a post due to lots of spelling mistakes, sms language, or special characters). We use Apache openNLP[9] to compute the part-of-speech tags of the questions [Church 1988].

4.1.1. Difficulty Level of a Question. Liu et al. [2013] used the TrueSkill model [Herbrich et al. 2007] to compute the difficulty of questions by considering question answering as a two-player competition where all the answerers win against the question asker and the best answerer wins against all other answerers. This model assumes that the best answerer has more skill than other answerers. However, the question-answering process is also about the timing of one's answer. Prior research shows that the first answer tends to accumulate more votes and have a higher probability of getting selected as the best answer [Pal and Konstan 2010]. The second answerer could be filling gaps in the first answer and hence a person of higher skill. Additionally, it is not clear how to use this model to assess the difficulty of a new question or an unanswered question.

To handle these aspects, we propose a novel model for estimating question difficulty. We use the following notation. Let x and y represent two distinct users. Let $\{q_{x1}, \ldots, q_{xn_x}\}$ be the questions asked by the user x. Let $w_{xiyj}(\geq 0)$ indicate similarity between the questions q_{xi} and q_{yj}. Let e_{xi} indicate the expertise of the best answerer to the question q_{xi}. We compute the difficulty (d) of the questions through the following minimization:

$$\text{minimize} \sum_{x} \sum_{i=1}^{n_x} \left\{ (d_{xi} - e_{xi})^2 + \lambda_1 \sum_{y \neq x} \sum_{j=1}^{n_y} w_{xiyj} |d_{xi} - d_{yj}| + \lambda_2 \sum_{j=1}^{n_x} |d_{xi} - d_{xj}| \right\} \tag{9}$$

subject to

$$d_{xi} \geq 0, \quad \forall x, \forall i,$$

where λ_1, λ_2 are the parameters that control solution sparsity (i.e., number of difficulty levels). Taking a cue from Liu et al. [2013], we aim at matching the difficulty level of a question to the expertise of the best answerer on that question. The second constraint aims at matching the difficulty level of similar questions. The third constraint aims at matching the difficulty level of questions asked by one user. Equation (9) is a convex optimization problem that can be solved efficiently by off-the-shelf optimization solvers [Grant and Boyd 2008].

For a new question q_{xk}, its difficulty level can be computed through the following minimization:

$$\text{minimize} \left\{ \lambda_1 \sum_{y \neq x} \sum_{j=1}^{n_y} w_{xkyj} |d_{xk} - d_{yj}| + \lambda_2 \sum_{j=1}^{n_x} |d_{xk} - d_{xj}| \right\} \tag{10}$$

subject to

$$d_{xk} \geq 0.$$

We set $\lambda_1 = \lambda_2 = \frac{1}{n}$, where $n = \sum_x n_x$ is the total number of questions. We compute question–question similarity (w) through cosine similarity between the questions' word vectors ($QC1$). Clearly, the minimization in Equation (10) can be seen as a tradeoff between the median difficulty of topical questions and the median difficulty level of the questions asked by the user.

[9]https://opennlp.apache.org/.

Table II. User Features

Type	Id	Feature
Basic Features	$UB1$	Questions answered by the user
	$UB2$	Questions asked by the user
	$UB3$	Demographics of user (job, department, location)
	$UB4$	Communities that user created or owns
	$UB5$	Communities that contain user as a member
Computational Features	$UC1$	Topical expertise of the user
	$UC2$	Word vector of questions answered by the user (normalized using Tf-Idf)
	$UC3$	Availability of the user
	$UC4$	Inclination to respond to another user
	$UC5$	Language analysis (e.g., nouns, pronouns, sentiment, politeness, openness)

4.2. User Features

Table II provides a complete list of user features. In order to compute users' topical expertise $UC1$, we consider a variant of *z-score* expertise model [Zhang et al. 2007] defined as $z\text{-}score = \frac{\#a - \#q}{\sqrt{\#a + \#q}}$, where $\#a$ is the number of answers given by the user and $\#q$ is the number of questions asked by the user. This simple scheme of estimating expertise has been found to be better than more complicated models like PageRank and HITS, as shown in Zhang et al. [2007]. To estimate the topical expertise, we consider topical *z-score* [Chang and Pal 2013] as follows:

$$z\text{-}score(t) = \frac{a(t) - q(t)}{\sqrt{a(t) + q(t)}}, \tag{11}$$

where t is a topic, $a(t) = \sum_{z \in UB1} z(t)$ indicates the sum of topic t's component for all the questions answered by the user, and similarly $q(t) = \sum_{z \in UB2} z(t)$ represents the sum of topic t's component of all the questions asked by the user. Note that topical *z-score* is a generalization of base *z-score*, as we can retrieve base *z-score* by setting the number of topics to 1. Apart from that, we also compute expertise through the first-answerer-based model proposed by Pal et al. [2012]. This model requires only limited data to identify if a user has the potential of expertise and is useful in cold-start situations.

User availability ($UC3$) has been used by prior work [Liu and Agichtein 2011] to find the most appropriate answerers to a question to reduce the expected waiting time of the asker. This is done by looking at the activity patterns of the users and detecting the amount of activity they perform per login session. We develop a similar probabilistic model by examining the most recent 2 weeks' hourly activity of the users and assigning a probability proportional to the frequency of activity per hour. We limit to 2 weeks only, at it provides a better estimate of users' current activity levels over their full historical activity.

Inclination to respond to a given user ($UC4$) is computed based on the strength of the shortest path between the two users in the QA graph. The QA graph [Zhang et al. 2007] is a directed weighted graph generated by connecting the question asker (x) to the answerer (y). The edge weight indicates the number of questions of x answered by y. Figure 1 shows a simple example of the QA graph, where C has answered A's question, A has answered D's question, and so on. We define the inclination to respond as the minimum weight on the shortest directed path from the asker to the potential answerer. If no path exists, then inclination is set to 0.5. If there is more than one shortest path, then we consider the maximum of the minimum weights on all the shortest paths. The intuition behind this feature is that if a user u has replied to another user v in the past

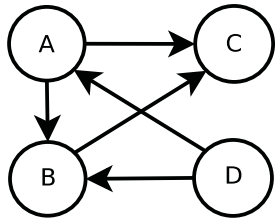

Fig. 1. Graph representation of question/answer threads.

Table III. Community Features

Type	Id	Feature
Basic Features	*CB*1	Members of the community
	*CB*2	Administrators of the community
	*CB*3	Demographics of community (job, department, location)
	*CB*4	Length of the questions asked in the community
	*CB*5	Difficulty level of the questions asked in the community
Computational Features	*CC*1	Word vector of community title and description
	*CC*2	Topical distribution of the community
	*CC*3	Topical expertise of the community members
	*CC*4	Availability of the community members
	CC5	Inclination to respond to another user
	*CC*6	Language analysis (e.g., nouns, pronouns, sentiment, politeness, openness)
	*CC*7	Community norms (typos, greetings, curse words, length, negative sentiment, etc.)

or if u connects to v with a high-strength directed path, then u might like to reply to v in the future as well. The intuitive justification of this hypothesis is that u and v share similar topical interest and/or they are familiar with one another.

4.3. Community Features

Table III presents a list of community features. The demographics of the community (*CB*3) are computed by taking the majority demographics of the community members and administrators (*CB*1 and *CB*2). The topical distribution (*CC*2) is computed using LDA over community documents (*CD*s). The rest of the computational features are computed by first considering that all the activity in a community is performed by one user. Then community features can be computed in a similar fashion as we compute individual features. Note that this feature computation strategy ignores the roles users play in a community.

5. SIMILARITY METRICS

For a routed question q, we compute its similarity with the three system entities. In this section, we present several similarity metrics computed based on the features listed in Tables I, II, and III.

5.1. Question–Question Similarity Metrics

First, we compute the topical similarity of the question q with the existing questions. In order to do this, we use Kullback-Leibler divergence (KL) defined as follows:

$$KL_{x||y} = \sum_i x_i * \log\left\{\frac{x_i}{y_i}\right\}, \tag{12}$$

where x_i and y_i are the i^{th} component of the probability distribution x, y. A low KL value indicates high similarity, and a high KL value indicates low similarity. We use the topic similarity $KL_{QC2_q||QC2_p}$ to identify relevant questions and then route q to the communities where these relevant questions were asked. However, we need to incorporate another important factor: how well those questions were answered. If those questions were not properly answered, then it might be counterintuitive to route q to their communities. To incorporate this factor, we propose the notion of question value $Val(p)$, which is estimated based on the number of people who viewed the question, votes they gave on the answers, and the number of unique users who answered it:

$$Val(p) = [QB1_p + QB4_p] \cdot \log(1 + QB2_p - QB3_p). \tag{13}$$

So an unanswered question has a value 0. A question that received five answers from two distinct users has a lower value ($5 \log 3$) than a question that received four answers from three distinct users ($4 \log 4$). Val tries to estimate the relative interest of the community toward answering a question and how well they answered the question. The topic match between the routed question q and an existing question p is then defined as

$$q\text{-}topic(q, p) = Cosine(QC4_q, QC4_p) - Val(p) \cdot KL_{QC2_q||QC2_p}, \tag{14}$$

where $Cosine(a, b) = \frac{a^T b}{||a||_2||b||_2}$ is the cosine similarity between two vectors a and b [Tan et al. 2005]. Note that we use cosine similarity here instead of KL because the part-of-speech tagging need not be a probability distribution. Note that we subtracted the KL score from the cosine similarity because a lower KL value is more desirable for a good match. Similarly, we compute the match between the language features of the two questions:

$$q\text{-}lang(q, p) = Val(p) \cdot Cosine(QC3_q, QC3_p). \tag{15}$$

We also compute whether the difficulty level of the routed question (computed using Equation (10)) matches that of an existing question:

$$q\text{-}diff(q, p) = |QC5_q - QC5_p|. \tag{16}$$

5.2. User– Question Similarity Metrics

Similar to question– question similarity metrics, we compute user– question similarity metrics. To do this, we first compute the familiarity of the potential answerers (u) with the question asker (v). This factor is computed based on the hypothesis that users who share a lot of communities, who have similar demographics, or who have interacted with one another previously might be more familiar with one another. Familiarity of the two users is defined as follows:

$$F(v, u) = \log \left\{ \begin{array}{l} 1 + UC4(v, u) + \alpha J(UB3_v, UB3_u) \\ \quad + \beta J(UB4_v, UB4_u) + \gamma J(UB5_v, UB5_u) \end{array} \right\}, \tag{17}$$

where α, β, γ are the weight parameters and $J(a, b) = \frac{|a \cap b|}{|a \cup b|}$ is the Jaccard index between two sets a and b [Rogers and Tanimoto 1960]. The motivation behind F is that the higher the familiarity is, the higher the chances of answering are. We consider three metrics for finding potential answerers:

$$u\text{-}exprt(q, u) = F(v, u) \cdot KL_{QC2_q||UC1_u}, \tag{18}$$

where $u\text{-}exprt$ estimates the topical expertise match of the user to the routed question;

$$u\text{-}lang(q, u) = F(v, u) \cdot Cosine(QC3_q, UC5_u), \tag{19}$$

where *u-lang* estimates the language match between the question answered by u' previously and the routed question; and

$$u\text{-}avail(q,u) = F(v,u) \cdot UC3(q,u), \tag{20}$$

where *u-avail* estimates the availability of the user around the time of the post of the question. The intuition behind multiplication of the metrics with familiarity is that the algorithms would then prefer moderate experts who are familiar to the asker in comparison to a top expert who doesn't know the asker at all. Note that we use log scaling in the familiarity term (F) to ensure that it doesn't dominate the three metrics.

5.3. Community Question Similarity Metrics

First, we estimate the topical match of q with a community c. To do this, we use the topic distribution of the question and the community. Additionally, a community contains a brief description supplied by its owners. This description could be useful as it might hold clues to the topical interest of the community. We combine these two factors to compute topical similarity as follows:

$$c\text{-}topic(q,c) = \delta \cdot Cosine(QC1_q, CC1_c) - KL_{QC2_q||CC2_c}, \tag{21}$$

where δ is the relative importance of community description over KL match. Now, similar to *u-exprt*, we compute the expertise of the community toward answering q as follows:

$$c\text{-}exprt(q,c) = KL_{QC2_q||CC3_c}. \tag{22}$$

In a similar fashion, we compute the *c-lang* and *c-avail*. Note that c-$*$ metrics are computed by considering all the activity within a community, ignoring the users who produced that activity.

We compute whether a question adheres to the community guidelines and norms by finding the *norm* dimension that the question violates the most. This is done as follows:

$$c\text{-}norm(q,c) = \max_{i=1}^{d} \left\{ (QC6_{c,i} - CC7_{c,i}) \cdot \mathcal{I}\left[(QC6_{c,i} \geq CC7_{c,i}) \right] \right\}, \tag{23}$$

where d is the dimensionality of *norm* ($QC6, CC7$) and $QC6_{q,i}$ indicates the i^{th} norm dimension of question q. $\mathcal{I}[cond]$ is the indicator random variable, which is 1 if the condition is true, otherwise 0.

6. RECOMMENDATION ALGORITHMS

In the previous section, we presented several similarity metrics that can be computed between the routed question and the different entities of the system. Here, we present algorithms for computing the community scores from these similarity metrics. Since c-$*$ similarity metrics already represent community scores, we are focused toward computing community scores from the q-$*$ and the u-$*$ similarity metrics.

6.1. Cutoff Aggregation Algorithm

We use the following notation: (1) *q is the routed question*, (2) *$\mathcal{C} = \{c_1, \ldots, c_m\}$ is the set of communities*, (3) *$\mathcal{O} = \{o_1, \ldots\}$ is the set of entities (questions or users)*, (4) *$n_{c,O} = \sum_{o \in O} o \in c$ is the number of objects in $O \subseteq \mathcal{O}$ belonging to community c*, (5) *$\mathcal{M} : q \times \mathcal{O} \rightarrow \mathbb{R}$ represents a similarity metric between the routed question and the objects in $\mathcal{O}$*, (6) *$\mathcal{W} = [w_1, \ldots, w_m]$ is the set of m object weight vectors (one per community), s.t. $|w_i| = n_{c,O}$*, and (7) *$\mathbb{F}$ is a function that takes in two variable-length vectors and produces a scalar.*

CUTOFF-AGGREGATION (CA) algorithm (Algorithm 1) picks the top-k most relevant objects to a question q. Relevance of an object to a question is quantified through the function $\mathcal{M}$. Then, for each community, it builds an ordered set of the top-k relevant objects that appeared in that community ($\mathcal{O}_s^k$). Then it considers the aggregation

ALGORITHM 1: CUTOFF-AGGREGATION $(k, \mathcal{W}, \mathcal{M}, \mathbb{F}, \mathcal{O}, \mathcal{C}, q)$

1: $k = \min\{k, |\mathcal{O}|\}$
2: $\mathcal{M}$ defines a partial ordering on the set $\mathcal{O}$. Consider the ordered set $\mathcal{O}_s = \{o_{i_1}, o_{i_2}, \ldots\}$, such that $\mathcal{M}(q, o_{i_a}) \geq \mathcal{M}(q, o_{i_{a+1}})$ for all a, $\forall o \in \mathcal{O} \Rightarrow o \in \mathcal{O}_s$.
3: Let $\mathcal{O}_s^k = \{o_{i_1}, o_{i_2}, \ldots, o_{i_k}\}$ represent the k-cutoff subset of $\mathcal{O}_s$. Here $\mathcal{O}_s^k$ is constructed by keeping the first k elements of $\mathcal{O}_s$ and discarding the rest.
4: **for** $c \in \mathcal{C}$ **do**
5: /* Construct a vector of similarities $\mathcal{M}(q, o)$ with objects $o \in \mathcal{O}_s^k$ that belong to the community c, such that vector elements are arranged in decreasing order */

$$M_c = \arg\max_M |M|$$

subject to

$$M = [o_{k_1} \;\; o_{k_2} \;\; \ldots \;]^T$$

$$o_{k_i} \mapsto c \;\wedge\; o_{k_i} \in \mathcal{O}_s^k \wedge \mathcal{M}(q, o_{k_i}) \geq \mathcal{M}(q, o_{k_{i+1}}), \forall i$$

6: Since $|M_c| \leq n_{c,\mathcal{O}}$, pad vector M_c with $n_{c,\mathcal{O}} - |M_c|$ zeros at the end.
7: Compute score, $CS_{\mathcal{M}}(c) = \mathbb{F}(w_c, M_c)$, where w_c is c^{th} component of $\mathcal{W}$.
8: **end for**
9: **return** $CS_{\mathcal{M}}$ similarity score of each community corresponding to similarity function $\mathcal{M}$.

function $\mathbb{F}$ over the ordered set of relevant objects per community ($\mathcal{O}_s^k$) to compute a community score. The aggregation function is the core of the *CA* algorithm, and in the following text, we will show how some simple yet popular aggregation strategies to compute group scores (such as *min*, *max*, *mean*, and *median*) can be modeled through the *CA* algorithm.

Definition 1 (*Simple Aggregation Strategy*). The simple aggregation strategy is to consider either the *min*, *max*, *mean*, or *median* functions to compute community scores from the entity similarity metrics.

(1) ***min*** strategy considers the similarity of the least matching object in the community.
(2) ***max*** strategy considers the similarity of the most matching object in the community.
(3) ***mean*** strategy considers the average similarity of all the objects in the community.
(4) ***median*** strategy considers the median similarity of all the objects in the community.

LEMMA 1. *Simple aggregation strategies can be modeled through the CA algorithm.*

PROOF. Let community c have $|c|$ objects. Consider the following weight vectors:

$$w_c^{min} = \Big[\; \underbrace{0 \;\ldots\; 0 \;\ldots\; 0}_{|c|-1} \;\; 1 \;\Big]^T$$

$$w_c^{max} = \Big[\; 1 \;\; \underbrace{0 \;\ldots\; 0 \;\ldots\; 0}_{|c|-1} \;\Big]^T$$

$$w_c^{mean} = \frac{1}{|c|}\Big[\; \underbrace{1 \;\ldots\; 1 \;\ldots\; 1}_{|c|} \;\Big]^T$$

$$w_c^{median} = \Big[\; \underbrace{0 \;\ldots\; 0 \;\ldots\; 0}_{\frac{|c|-1}{2}} \;\; 1 \;\; \underbrace{0 \;\ldots\; 0 \;\ldots\; 0}_{\frac{|c|-1}{2}} \;\Big]^T.$$

ALGORITHM 2: GLOBAL-KNN($k, \mathcal{M}, \mathcal{O}, \mathcal{C}, q$)

1: /* Construct $\mathcal{W}$ vectors */
2: $w_c = [\ \underbrace{1\ \ \ldots\ \ 1\ \ \ldots\ \ 1}_{n_{c,\mathcal{O}}}\ ]^T$, for all $c \in \mathcal{C}$
3: /* Define $\mathbb{F}$ as count of nonzero elements in the dot product of two vectors */
4: $\mathbb{F}_{count}(\vec{a}, \vec{b}) = \sum_i \mathcal{I}[\vec{a}_i \cdot \vec{b}_i > 0]$, where $\mathcal{I}$ is indicator random variable
5: **return** CUTOFF-AGGREGATION ($k, \{w_c\}_{c\in\mathcal{C}}, \mathcal{M}, \mathbb{F}_{count}, \mathcal{O}, \mathcal{C}, q$)

Consider $\mathbb{F}_{dot}(\vec{a}, \vec{b}) = \vec{a}^T\vec{b}$ as the function that computes the dot product of two vectors. We note that CUTOFF-AGGREGATION $\left(\infty, \mathcal{W} = \{w_c^{min}\}_{c\in\mathcal{C}}, \mathcal{M}, \mathbb{F}_{dot}, \mathcal{O}, \mathcal{C}, q\right)$ models the *min* aggregation strategy. Similarly, passing weights $\{w_c^{max}\}_{c\in\mathcal{C}}$, $\{w_c^{mean}\}_{c\in\mathcal{C}}$, $\{w_c^{median}\}_{c\in\mathcal{C}}$ to the *CA* algorithm models *max*, *mean*, *median*, respectively. □

6.2. Instantiation of CUTOFF-AGGREGATION ALGORITHM

We present two k-nearest-neighbor (*knn*) algorithms that are a natural instantiation of the CUTOFF-AGGREGATION algorithm. The intuition behind these *knn* algorithms is that *min*-, *max*-, *mean*-, and *median*-based aggregation strategies do not capture the true characteristics of a community—primarily due to the skew in the activity levels of the community members. The two algorithms take as input the following parameters: number of nearest neighbors k, a question q, a set of objects $\mathcal{O}$, a set of communities C that contain the objects in $\mathcal{O}$, and a similarity metric $\mathcal{M}$ between q and the objects in $\mathcal{O}$.

GLOBAL-KNN. Algorithm 2 presents the GLOBAL-KNN algorithm to rank communities. It first picks the k most relevant objects based on their $\mathcal{M}(\cdot)$ score. Then it computes the score of each community based on how many of its objects appear in the top-k list. Note that it differs from the *mean* aggregation strategy in three ways: (1) the weights are not normalized, (2) k is not set to ∞, and (3) each object in top k contributes 1 to the community score, irrespective of its similarity to the routed question.

LOCAL-KNN. Algorithm 3 presents the LOCAL-KNN algorithm. It picks the top-k most similar objects per community and takes their average similarity as the community score. One issue with this approach is that it can be biased toward communities with less than k objects.

LEMMA 2. *Let c_1 and c_2 be two communities with x and y objects, respectively. Consider $x \le k_1 < k_2 \le y$. If the nearest neighbor parameter k is varied from k_1 to k_2, then in the worst case, the* LOCAL-KNN *algorithm reduces the community score of c_2 by a factor of $\frac{k_1}{k_2}$ while keeping the score of c_1 constant.*

PROOF. Consider $k_1 \le k \le k_2$. $n_{c_1} = min(k, x) = x$ for all k in k_1 to k_2. So the score of community c_1 remains constant. However, $n_{c_2} = min(k, y) = k$ because $k \le y$. Let the

ALGORITHM 3: LOCAL-KNN($k, \mathcal{M}, \mathcal{O}, \mathcal{C}, q$)

1: /* Construct $\mathcal{W}$ vectors */
2: **for** $c \in \mathcal{C}$ **do**
3: $\quad n_c = \min(k, n_{c,\mathcal{O}})$
4: $\quad m_c = \min(0, n_{c,\mathcal{O}} - k)$
5: $\quad w_c = \frac{1}{n_c}[\ \underbrace{1\ \ \ldots\ \ 1\ \ \ldots\ \ 1}_{n_c}\ \ \underbrace{0\ \ \ldots\ \ 0\ \ \ldots\ \ 0}_{m_c}\ ]^T$
6: **end for**
7: **return** CUTOFF-AGGREGATION ($\infty, \{w_c\}_{c\in\mathcal{C}}, \mathcal{M}, \mathbb{F}_{dot}, \mathcal{O}, \mathcal{C}, q$)

score of c_2 for $k = k_1$ be s. The score of c_2 for $k = k_2$ is $\frac{k_1 \cdot s + (k_2 - k_1) \cdot \bar{s}}{k_2}$, where $\bar{s} \leq s$. The reduction in score is by a factor of $\frac{k_1}{k_2} + (1 - \frac{k_1}{k_2})\frac{\bar{s}}{s}$. Clearly, in the worst case ($\bar{s} = 0$), the reduction is $\frac{k_1}{k_2}$. In the best case ($\bar{s} = s$), there is no reduction. □

Lemma 2 states that a community with a large number of objects would be relatively less preferable for large k. To tackle this problem, we use a log-based normalization of the community scores. This is achieved by the following function:

$$\mathbb{F}_{log}(\vec{a}, \vec{b}) = \mathbb{F}_{dot}(\vec{a}, \vec{b}) \cdot \log\{\mathbb{F}_{count}(\vec{a}, \vec{b})\}, \tag{24}$$

where $\mathbb{F}_{dot}(\vec{a}, \vec{b}) = \vec{a}^T \vec{b}$, and $\mathbb{F}_{count}(\vec{a}, \vec{b}) = \sum_i \mathcal{I}[\vec{a}_i \cdot \vec{b}_i > 0]$. Log-based normalization ensures that the communities with a small number of objects are penalized by a factor of $\frac{\log x}{\log k}$. This penalization mitigates the effect of the algorithm's bias. Note that if we remove the log function and simply use $\mathbb{F}_{dot} \cdot \mathbb{F}_{count}$, then it would create a bias toward bigger communities.

6.3. Community Ranking Algorithm

We use the two *knn* algorithms over the similarity metrics to build several community scores. We collect all these scores for a community in a score vector $\vec{f}_{c,q}$. Now the problem is to use $F_q = [\vec{f}_{c,q}]^T, c \in \mathcal{C}$ to rank communities. To do this, we consider three mechanisms.

6.3.1. Linear Regression. The first mechanism is to use *linear regression* for ranking [Kumar et al. 2011; Hong et al. 2012]. We first construct a binary response variable (y), which is set to 1 for the desired community and 0 for the nondesired ones. Then, optimal weights w_{lr} are estimated using the linear regression framework. The closed-form solution of the linear regression is $w_{lr} = (F_q^T F_q)^{-1} F_q^T Y$, where F_q is the design matrix and $Y = [y_{c,q}]_{c \in \mathcal{C}}^T$ is the output response. The weight vector w_{lr} indicates the relative importance of different community scores in estimating the desirability of communities. For final ranking, we sort communities in decreasing order of their $w_{lr}^T \cdot \vec{f}_{c,q}$ value.

6.3.2. Borda Count. The second mechanism is to generate a ranked list per score type and then merge those ranked lists. The general version of this problem is NP-hard, but there are several greedy algorithms [Fagin et al. 2003]. We consider a simple yet popular *Borda count* algorithm [Coppersmith et al. 2006]. The algorithm simply computes the aggregate rank of a community and then sorts on their aggregate rank. We consider a weighted version of the algorithm, in which weighted aggregate rank is computed. In order to learn the weights, we use an iterative reweighing scheme, where the weight of one ranked list is estimated by fixing the weights of other ranked lists, and so on. The weights that minimize the rank of desired communities are chosen.

6.3.3. SVM Ranking. The third mechanism is to cast it as a convex optimization problem with pairwise constraints. We use *ranking SVM*, which is an extremely popular algorithm for learning from ranked lists [Joachims 2002]. Let R_q and S_q be the set of desired and nondesired communities for a question q. Then, the optimal weights w_{svm}

are obtained through the following minimization:

$$\begin{aligned} \textit{minimize}: \quad & \frac{1}{2} w^T w + \lambda \sum \xi_{a,b,q} \\ \textit{subject to}: \quad & \\ \forall q, \forall a \in R_q, \forall b \in S_q: \quad & w^T (\vec{f}_{a,q} - \vec{f}_{b,q}) > 1 - \xi_{a,b,q} \\ \forall q, \forall a \in R_q, \forall b \in S_q: \quad & \xi_{a,b,q} \geq 0. \end{aligned}$$

The constraints are geared toward finding a clear separation between the desired and the nondesired set of communities. The ξ indicates soft-margin penalties to the separation constraints without which the minimization problem might be infeasible (in case the two sets of communities are not linearly separable). For final ranking, we sort communities in decreasing order of their $w_{svm}^T \cdot \vec{f}_{c,q}$ value.

6.4. Pipeline Algorithm

We can combine the ranking algorithms in a pipeline fashion. Suppose we need a ranked list of size n; then we can run the first algorithm to get a ranked list of size $5n$, and then feed this as input to the second algorithm and get a ranked list of size n. The first model acts as a filter to eliminate the noise and the second can then produce a reliable ranked list. Our results show that the pipeline model can be more effective than running the models separately.

6.5. Partitioning Algorithms

The CUTOFF-AGGREGATION algorithm cycles through all the entities to compute the community scores. As the number of entities becomes large, it can be prohibitive for real-time scenarios. In order to reduce the runtime complexity, we cluster the communities into l clusters. To do this, we compute the community– community affinity by computing their topic similarity ($KL_{CC2_{c_1}||CC2_{c_2}}$) and then running a spectral clustering algorithm using the normalized cut criteria [Dhillon et al. 2004]. For routing a question, we pick the most topically appropriate cluster and run the recommendation algorithms for the communities within that cluster. This strategy leads to a best-case reduction in time complexity by a factor of l.

6.6. Complexity Analysis

The worst-case time complexity of the *CA* algorithm is $\mathbb{O}(|\mathcal{O}| \cdot \log |\mathcal{O}| + |\mathcal{C}| \cdot T_{\mathbb{F}})$. This is because the construction of $\mathcal{O}_s^k$ requires sorting $|\mathcal{O}|$ objects. Computation of the community scores from these sorted objects is $T_{\mathbb{F}}$ in the worst case, where $T_{\mathbb{F}}$ is the time complexity of $\mathbb{F}(x, y)$, with x and y as two arbitrary vectors of length $|\mathcal{O}|$—for $\mathbb{F}_{dot}$ and $\mathbb{F}_{count}$, this is simply $\mathbb{O}(|\mathcal{O}|)$. So *CA* has a worst-case time complexity of $\mathbb{O}(|\mathcal{O}| \cdot \log |\mathcal{O}| + |\mathcal{C}| \cdot |\mathcal{O}|)$ for the two scoring functions ($\mathbb{F}$). Note that typically $|\mathcal{C}|$ is much smaller than $|\mathcal{O}|$, so the first term dominates the running time.

LEMMA 3. *Let $\mathcal{W}_{binary} = \{w\}$ be a set of binary object weight vectors, which obeys the following conditions:*

$$\begin{aligned} w_i = 1 &\Rightarrow w_j = 1, \forall j < i \\ w_i = 0 &\Rightarrow w_j = 0, \forall j > i. \end{aligned}$$

Let $\mathbb{F}' \in \{\mathbb{F}_{dot}, \mathbb{F}_{count}\}$. The worst-case time complexity of the function call CUTOFF-AGGREGATION *$(k, \mathcal{W}_{binary}, \mathcal{M}, \mathbb{F}', \mathcal{O}, \mathcal{C}, q)$ is $\mathbb{O}(|\mathcal{O}| + k \cdot |\mathcal{C}|)$ in the worst case.*

PROOF. Instead of sorting the k objects on their similarity scores, we simply pick the top-k objects. This can be done in time $\mathbb{O}(|\mathcal{O}|)$ by using the Selection algorithm

[Blum et al. 1972]. These k objects can belong to all the $|\mathcal{C}|$ communities. Now let $n_w = w^T\ 1$ be the number of 1^s in the binary weight vector, where *1* is a vector with $|w|$ components with all 1. Let $\bar{n}_w = \min(n_w, k)$. Clearly, we need to find $\bar{n}_{w_c}$ top-matching objects in community c from k objects. These objects can be identified in $\mathbb{O}(k)$ time. Once we retrieve these objects, we can run the $\mathbb{F}'$ scoring function, which takes $\mathbb{O}(\bar{n}_{w_c})$ time. So the total time complexity of the algorithm is $\mathbb{O}(|\mathcal{O}| + \sum_c \bar{n}_{w_c})$. We know that $\sum_c \bar{n}_{w_c} \leq \sum_c k = k \cdot |\mathcal{C}|$. Hence, we get the worst-case running time of $\mathbb{O}(|\mathcal{O}| + k \cdot |\mathcal{C}|)$. Since $k \leq |\mathcal{O}|$, the worst-case running time is $\mathbb{O}(|\mathcal{C}| \cdot |\mathcal{O}|)$. □

Using Lemma 3, we see that the running time of the two *knn* algorithms is $\mathbb{O}(|\mathcal{O}| + k \cdot |\mathcal{C}|)$. For small k, it is simply $\mathbb{O}(|\mathcal{O}|)$.

The linear regression ranking algorithm runs in $\mathbb{O}(|\mathcal{C}|)$ time. The design matrix F_q has $|\mathcal{C}|$ rows and c columns, where c is the number of different similarity metrics. The term that dominates the time complexity is the computation of $F_q^T F_q$, which is $\mathbb{O}(c^2 \cdot |\mathcal{C}|)$.

The Borda count ranking algorithm runs in $\mathbb{O}(|\mathcal{C}| \cdot \log\{|\mathcal{C}|\})$ time. To see this, we sort the communities c times for each similarity measure. Then merging of the communities and getting the top k can be efficiently implemented through a hash-map and a min-heap.

For the SVM ranking algorithm, there are $|\mathcal{C}|$ constraints. This is because typically there is only one desired community and the rest are undesired ones. Clearly, the complexity of the algorithm is lower bounded by $\mathbb{O}\{|\mathcal{C}|\}$. The actual complexity depends on the algorithm used for the convex minimization, analysis of which is beyond the scope of this article.

7. DATASET

We experimentally evaluate the performance of our model on two datasets. These two datasets differ from one another on functional (practices, enterprise vs. online) as well as topical aspects. Experiments on these diverse datasets validate the wide applicability of our community routing framework.

7.1. Enterprise Dataset

We crawled an enterprise dataset from the internal IBM Connections website. IBM Connections consists of tens of thousands of communities specific to certain goals, such as socialization, collaboration, knowledge sharing, and increasing employee productivity. The breadth of topics in the communities varied from development and coding to finance and human resources to fun and outings. The community software allows community members to post questions and answers. By default, the communities are marked as public, which allows nonmembers to read the community data. We randomly selected a set of 2,087 public communities and crawled all their data. The crawled data consists of 59,561 questions asked by 44,318 users.

7.2. WebMD Dataset

We crawled featured communities listed at *exchanges.webmd.com*. These communities belonged to seven categories: *eating and diet*, *digestive disorder*, *men's health*, *women's health*, *parenting*, *pregnancy*, and *trying to conceive*. For each of these seven categories, we crawled 15 featured communities, which resulted in 105 communities overall. The dataset contained 53,430 unique users and 149,444 posts. Most of the crawled communities are competing with one another in the topic space. So it can be hard for end-users to figure out the right community for their question. For example, if a person has a technical question regarding conceiving, then there are several relevant communities such as *clomid exchange*, *polycyst ovary syndrome*, *12 month still trying group*, and so forth. A routing mechanism that could suggest a relevant community would save the user considerable time and effort.

8. EVALUATION

The goal of our evaluation is threefold:

(1) We want to evaluate the effectiveness of our model, which combines community features and individual member features, in comparison with prior work in group recommendation.
(2) We want to examine the effectiveness of the two *knn* algorithms proposed in this article and measure their performance for different choices of parameter k as well as for different settings of the topic models. We want to understand which features are most useful for community routing, so that those aspects of the communities can be provided more attention.
(3) We also want to test our hypothesis that a pipelining strategy to rank, where in one the ranking algorithm is used as filter and other is used as ranker, leads to more accurate results. We also want to test the effectiveness of the partitioning algorithm in speeding up the model.

For model evaluation, we use 10-fold cross-validation. In each fold, we consider 80% data for training, 10% for parameter estimation (holdout), and 10% for testing. In order to select the model parameters (α, β, γ, δ), we first generate several choices of the parameters, randomly. For each choice of the model parameters, the ranking models are trained using the training data and their performance measured over the holdout data. The parameter set that leads to the best performance is selected. This is done in order to avoid overfitting over the training dataset. Finally, the performance of the model is recorded on the test dataset. For a question in the test dataset, the task is to retrieve the community where that question was asked. Note that for each question, there is only one correct community. We also ensured that only well-answered questions ($Val(\cdot) > 0$; see Equation (13)) are considered. This filtering criterion led to a selection of 48,124 questions for the Enterprise data and 128,953 posts for the WebMD dataset.

We consider two evaluation metrics for assessing the routing algorithms. The first measure is precision at position N (P@N), which is 1 if the desired community is among the top N communities, otherwise 0.[10] The second measure is Mean Reciprocal Rank (MRR), which is the inverse of rank of the correct community averaged for all question queries.[11] These two measures are widely used in the IR domain [Manning et al. 2008].

8.1. Model Comparison

In order to run our model, we consider the best settings for the *knn* algorithms. To get the best settings, we divide the dataset into three parts: training, holdout, and testing. The model is trained on first part, and the parameters that perform best on the holdout are selected. Finally, the accuracy of the model on the test dataset is reported. This ensures that the generalization accuracy is reported for the models. Performances of the other models are also reported on the test datasets for an unbiased comparison.

We test the performance of the our three models described in Section 6.3. The three models are *linear regression*, *Borda count* and *ranking SVM*. Apart from these three models, we also report the performance of the following baseline models:

(1) **B1**. This model generates a random permutation of communities for a question.
(2) **B2**. This model sorts communities in order of their decreasing sizes, so it is sensitive toward communities with lots of objects.

[10]P@N is a slight variant of traditional precision. This variant is more suitable in our setting because there is only one correct community for a routed question.
[11]Note that since there is only one correct community per routed question, MRR is more suitable than Mean Average Precision (MAP).

Table IV. Performance of the Ranking Models

Dataset	Model	*P*@1	*P*@5	*MRR*
Enterprise	*Linear regression*	0.57	0.77	0.68
	Borda count	0.59	0.82	0.68
	Ranking SVM ($\lambda = 1$)	**0.64**	**0.84**	**0.73**
	B1: random	0	0.003	0.004
	B2: large	0.08	0.23	0.16
	B3: expertise	0.39	0.60	0.49
	B4: competition-expertise	0.37	0.56	0.44
	B5: content+collab filtering	0.51	0.79	0.66
WebMD	*Linear regression*	0.52	0.81	0.63
	Borda count	0.53	0.82	0.65
	Ranking SVM ($\lambda = 1$)	**0.55**	**0.85**	**0.68**
	B1: random	0.01	0.05	0.05
	B2: large	0.12	0.19	0.20
	B3: expertise	0.41	0.73	0.51
	B4: competition-expertise	0.39	0.71	0.49
	B5: content+collab filtering	0.49	0.76	0.58

(3) **B3**. This model selects the topmost expert and picks his or her communities. If the expert belongs to multiple communities, then ties are broken based on the topic match of the routing question with those communities. This is similar to the *Least-Misery* approach used for the group recommendation problem.
(4) **B4.** This model uses Liu et al. [2011] to compute the expertise of users. Then we consider the two aggregation strategies, *Average* and *Least-Misery*, to pick the top communities. The result of the best strategy is reported here.
(5) **B5.** This model is based on the content- and collaborative-filtering-based approach for group recommendation [Ye et al. 2012]. Apart from users' activity, this model requires users' social networks. For the Enterprise data, this is readily available. To infer the social graph for the WebMD dataset, we consider an edge between two users if they are members of the same community.

We selected the *B*1 and *B*2 baseline as they are very naive and quite straightforward to code. A side-by-side performance comparison of these "code easy" approaches with ours can be useful for system builders in deciding whether our approach (which can require more intensive coding) is worth it or not. Additionally, we selected the baselines *B*4 and *B*5, which are prior state-of-the-art approaches, for expertise finding and group recommendation, respectively. These baselines can operate on our underlying datasets and they provide a good benchmark for performance comparison.

Table IV shows the performance of our models in comparison with the several baselines. We observe that our models, which combined several different aspects of a community, perform significantly better than the models that route a question to the community of best individuals or a group (through collaborative filtering) to answer that question, using one-sided *t-test* ($p \sim 0$). One intuitive reason for this result is that our model captures several different aspects of a community and the baseline models are more focused on the expertise aspect of the community and the individuals.

Among the ranking models, the *ranking SVM* model performs significantly better than all other models using one-sided *t-test* ($p \leq 0.01$). Intuitively this makes sense, because the pair-wise constraints lead to a direct optimization of the *P*@1 accuracy measure, with some relaxation due to the ξ terms. On the other hand, *linear regression* suffers due to a rank deficit of the $F_q^T F_q$ matrix for both datasets. To make the matrix

Table V. Performance of the *knn* Algorithms over *Enterprise Dataset*

	Global-Knn			Local-Knn		
Metrics	*P*@1	*P*@5	*MRR*	*P*@1	*P*@5	*MRR*
q-topic	0.45	0.57	0.53	0.47	0.65	0.53
q-lang	0.18	0.25	0.20	0.19	0.33	0.24
q-diff	0.11	0.20	0.16	0.13	0.26	0.20
u-exprt	0.39	0.62	0.50	**0.48**	**0.66**	**0.55**
u-lang	0.13	0.32	0.21	0.24	0.41	0.31
u-avail	0.03	0.07	0.08	0.09	0.17	0.10
c-topic	0.38	0.54	0.43	0.38	0.54	0.43
c-exprt	0.29	0.46	0.34	0.29	0.46	0.34
c-norm	0.11	0.19	0.17	0.11	0.20	0.18
c-lang	0.09	0.18	0.15	0.09	0.18	0.15
c-avail	0.02	0.16	0.12	0.02	0.16	0.12

inversion numerically stable, we use *ridge regression,* where a regularization term cI is added to the rank-deficit matrix, where I is the identity matrix and $c = 10^{-5}$ is a small constant. This addition makes the problem numerically stable but did not improve the performance. Despite that, the adjusted r^2 for the linear regression model is 0.48 and 0.40 for the Enterprise and the WebMD dataset, respectively, which indicates a reasonable fit. The weighted *Borda count* method performs better than *linear regression* as it optimizes the final ranked list, but not as good as *ranking SVM* due to its weight update mechanism.

From Table IV, we observe that the performance of our model over the Enterprise dataset is better than over the WebMD dataset. Since the WebMD dataset has fewer communities, it is expected that the model should perform better for this dataset, which is precisely the case for the random model. One reason for this result is that the WebMD dataset has much higher topic overlap between the communities, as the disease symptoms are similar for several diseases. As a result, more communities qualify as candidates for routing.

Overall, we conclude that the combined models improve the performance beyond that achieved by running the models over individual metrics such as user expertise. Among the combined models, *ranking SVM* is significantly better for the two datasets.

8.2. Model + Metric Evaluation

Our model is composed of several similarity metrics and their aggregation through the two *knn* algorithms. We begin a systematic evaluation of these critical components of our model. To run these experiments, we set the number of nearest neighbor (k) to 5. We also set the number of topics to 150 and 30 for the Enterprise and WebMD datasets, respectively. Tables V and VI show the performance of the *knn* algorithms over several similarity metrics for the two datasets.[12] We make the following key observations from Tables V and VI:

—Similarity metrics *q-topic* and *u-exprt* perform significantly better than all other similarity metrics across all the conditions using one-sided *t-test* ($p \sim 0$). *q-topic* measures the average similarity of the routed question with the existing questions in the community, and *u-exprt* estimates the average topical expertise of community members. Among the two metrics, *u-exprt* is significantly better for the Enterprise dataset and *q-topic* is slightly better for the WebMD dataset.

[12]Ties between communities are broken based on the topic match of the routed question with the communities.

Table VI. Performance of the *knn* Algorithms over *WebMD Dataset*

	Global-Knn			Local-Knn		
Metrics	*P*@1	*P*@5	*MRR*	*P*@1	*P*@5	*MRR*
q-topic	0.43	0.63	0.52	**0.45**	**0.78**	**0.58**
q-lang	0.15	0.36	0.35	0.22	0.48	0.32
q-diff	0.07	0.21	0.17	0.09	0.24	0.19
u-exprt	0.42	0.64	0.58	0.44	0.77	0.57
u-lang	0.19	0.42	0.31	0.26	0.48	0.34
u-avail	0.02	0.12	0.14	0.05	0.18	0.16
c-topic	0.34	0.68	0.47	0.34	0.68	0.47
c-exprt	0.31	0.64	0.45	0.31	0.64	0.45
c-norm	0.22	0.49	0.36	0.22	0.50	0.38
c-lang	0.19	0.46	0.30	0.19	0.46	0.30
c-avail	0.03	0.16	0.12	0.03	0.16	0.12

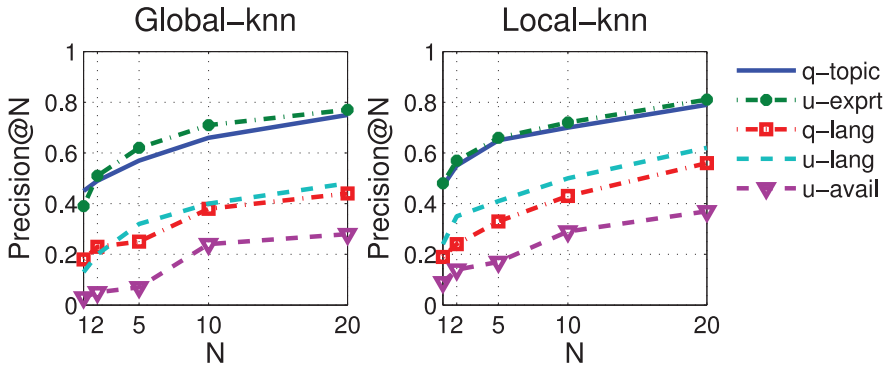

Fig. 2. Precision@N for the Enterprise dataset.

—The Local-Knn algorithm is significantly better than Global-Knn for all similarity metrics using one-sided *t-test* ($p \leq 0.01$). It indicates that computation of community scores based on the top-k users (questions) within a community is more effective than first picking the top-k users (questions) globally and then computing the community scores through a voting mechanism. Note: Local-Knn leads to a within-community top-k selection as it passes $k = \infty$ as a parameter to the Cutoff-Aggregation algorithm.
—Similarity metrics over community features (c-$*$), which are computed by aggregating all activity within a community naively without considering semantics of users and questions, performs significantly worse than their q-$*$ and u-$*$ counterparts across all conditions using one-sided *t-test* ($p \sim 0$). This result highlights that a naive merging of all the activity ignores the roles that different users play in a community and fails to capture their characteristics' distribution, which *knn* algorithms capture effectively through the k parameter.
—The performance of the *q-topic* metrics improved from our prior work. This is primarily because of the addition of number of votes in the assessment of a question value and also in part due to the part-of-speech matching. The improvement is significant for *P*@5 ($p < 0.025$).

Figure 2 plots *P*@*N* for u-$*$ and q-$*$ similarity metrics. We observe an increase in precision by 70% to 150% from $N = 1$ to $N = 20$. The precision tapers off for large N.

Table VII. Best Performance of Simple Aggregation Strategies

	Enterprise Dataset			WebMD Dataset		
Strategy	*P*@1	*P*@5	*MRR*	*P*@1	*P*@5	*MRR*
min	0.02	0.09	0.07	0.03	0.11	0.06
mean	0.31	0.55	0.41	0.37	0.70	0.46
median	0.21	0.36	0.27	0.26	0.42	0.33
max	**0.39**	**0.60**	**0.49**	**0.41**	**0.73**	**0.51**

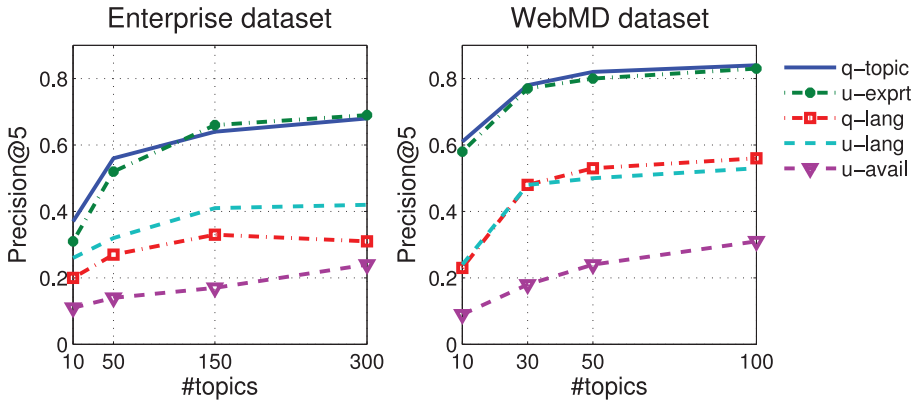

Fig. 3. Precision@5 of the LOCAL-KNN algorithm for different choices of number of topics.

We also note that if a metric is better than another metric, it is consistently better for all N. This consistency indicates high reliability of these metrics.

8.2.1. Performance of Simple Aggregation Strategies. Table VII reports the best performance of the simple aggregation strategies over different similarity metrics. The best performance is achieved over *q-topic* and *u-exprt* similarities. This result shows that *max* has the best performance among simple aggregation strategies. However, it is significantly worse than the best *knn* performance (Table V and VI).

8.2.2. Topic + Similarity Function Evaluation. Here, we evaluate the effectiveness of the topic models toward model performance. Note that the first step in computation of the similarity metrics is the inference of the topic distribution of the questions. So far we used the LDA topic model to run our experiments. One of the key parameters required by LDA is the number of topics. Here, we explore how the number of topics affects the performance of the models. We also evaluate other topic models such as LSA and the vector space model and different similarity functions such as KL divergence, Cosine similarity, and Jaccard index.

Figure 3 shows the precision of the LOCAL-KNN algorithm for several choices of number of topics (#topics). It shows that an increase in #topics leads to an increase in the performance. This increase is large for small values (around 50%) and is small for large values (around 1%–2%). Intuitively, a small value of #topics leads to underfitting in the topic space, which in turn leads to an increase in false positives. On the other hand, a large value of #topics could lead to overfitting, which in turn might eliminate desirable communities from the candidate set. Additionally, a large #topics leads to more iterations of the LDA algorithm to converge. Based on this experiment, we choose #topics $= 150, 30$ for the Enterprise and the WebMD dataset, respectively. Overall, this result shows that the models can be somewhat sensitive to the choice of number of topics.

Table VIII. Overall Performance of the Top-Performing Similarity Metrics Computed Using Various Topic and Similarity Scheme (we used LOCAL-KNN to compute these numbers)

			Enterprise			WebMD		
Metric	Topic	Similarity	*P*@1	*P*@5	*MRR*	*P*@1	*P*@5	*MRR*
q-topic	LDA	KL	0.47	0.65	0.53	**0.45**	**0.78**	**0.58**
	Tf-idf	Cosine	**0.48**	**0.71**	**0.57**	0.42	0.70	0.51
	VSM	Jaccard	0.47	0.70	0.55	0.39	0.71	0.55
u-exprt	LDA	KL	0.48	0.66	0.55	**0.44**	**0.77**	**0.57**
	Tf-idf	Cosine	**0.49**	**0.73**	**0.58**	0.38	0.71	0.53
	VSM	Jaccard	0.47	0.67	0.56	0.37	0.68	0.50
c-topic	LDA	KL	0.37	0.54	0.43	**0.34**	**0.68**	**0.47**
	Tf-idf	Cosine	**0.39**	**0.62**	**0.51**	0.33	0.68	0.47
	VSM	Jaccard	0.37	0.60	0.50	0.28	0.64	0.47

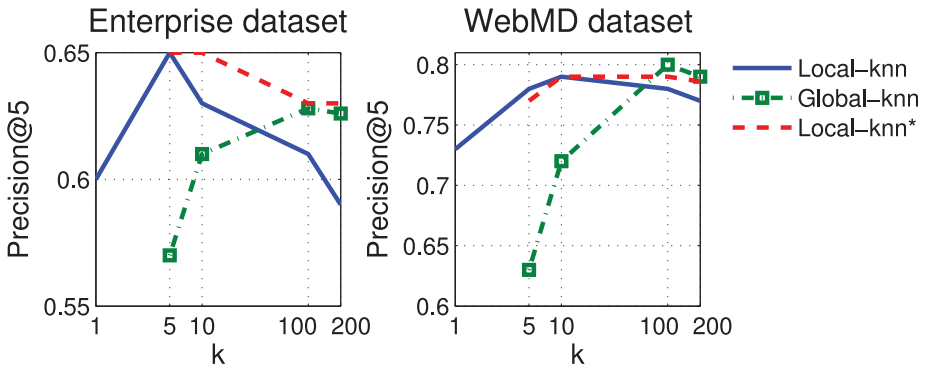

Fig. 4. Precision of the two *knn* algorithms over *q-topic* similarity metric. *Local-knn∗* represents the *Local-knn* algorithm with log scaling.

We explore two alternate topic representation schemes: Tf-idf based and vector space model (VSM). We consider three similarity functions: KL divergence, Cosine similarity, and Jaccard index. Table VIII shows the performance of different schemes of computing the top-performing similarity metrics. For the enterprise dataset, Tf-idf normalization along with Cosine similarity performs significantly better than the LDA- or VSM-based topic representations using one-sided *t-test* ($p \leq 0.015$). However, for the WebMD dataset, LDA-based topic representation outperforms alternate topic schemes using one-sided *t-test* ($p \leq 0.01$). Slightly worse performance of LDA over Tf-idf for the Enterprise dataset is counterintuitive, considering the fact that LDA is the current state of the art. We tried several different parameter configurations and ran LDA for much larger iterations, yet it did not fare better. Intuitively, LDA is a sparsity-inducing method, which assumes that a document consists of few topics. Additionally, it attempts to compute topics that are orthogonal to one another. However, in an enterprise, such clear topic separation is not available. Hence, a Tf-idf model that does not assume any latent factors works well. On the other hand, WebMD has clearly defined topics that do not overlap considerably. As a result, we see these differences in the performance. Overall, we conclude that the models can be sensitive to the choice of topic model and the number of topics.

8.2.3. Number of Nearest Neighbors. An important parameter required by the *knn* algorithms is the number of nearest neighbors k. Figure 4 shows their performance for different values of k. We note that $k = 1$ leads to a *max*-based aggregation and $k = \infty$ is *mean*-based aggregation. Both these aggregations appear to be significantly

Table IX. Performance of the Model After Removing One Feature at a Time

	Enterprise		WebMD	
Metrics	*MRR*	*%drop*	*MRR*	*%drop*
q-topic	0.65	10.96	0.61	10.30
q-lang	0.67	8.22	0.68	0
q-diff	0.72	1.37	0.67	1.47
u-exprt	0.61	16.44	58	14.71
u-lang	0.68	6.85	0.67	1.47
u-avail	0.73	0	0.68	0
c-topic	0.68	6.85	0.65	4.41
c-exprt	0.68	6.85	0.64	5.89
c-norm	0.71	2.74	0.67	1.47
c-lang	0.72	1.37	0.68	0
c-avail	0.73	0	0.68	0

worse in comparison to $k = 5$, highlighting that trivial aggregation mechanisms are ineffective.

We observe that the performance of GLOBAL-KNN increases and then ultimately decreases. The increase makes sense because as k is increased, the score of a topically relevant community with a lot of similar questions would increase. However, as k becomes large, the performance degrades as the algorithms get biased toward communities with a lot of questions irrespective of their similarity.

We observe a similar trend for the LOCAL-KNN algorithm. However, here, the performance starts to drop for $k > 5$. This is because as k is increased, the algorithm gets biased toward communities with fewer questions (see Lemma 3). To demonstrate this bias, consider a toy example. Say community c_1 has 10 questions with $q\text{-}topic = \{1, 1, 1, 1, .1, .1, .1, .1, .1, .1\}$ and community c_2 has two questions with $q\text{-}topic = \{.5, .5\}$. For $k = 5$, c_1 is preferred over c_2, whereas for $k = 10$, it is the other way around. This bias toward smaller communities is exacerbated as k is increased further. To solve this problem, we multiply the community scores with $\log(\min\{k, \#\text{object}\})$ (see Equation (24)). Figure 4 (*Local-knn*$*$) shows that the performance of log scaling remains steady, unlike its unscaled counterpart.

Overall, we see that for $k \leq 10$, LOCAL-KNN performs significantly better than GLOBAL-KNN. However, as we increase k beyond 100, GLOBAL-KNN outperforms *Local-knn*. Note that as k approaches ∞, the performance of the two algorithms converges (to a small value). The choice of k plays a crucial role in the performance of the algorithms.

8.2.4. Relative Importance of Similarity Metrics. A previous experiment shows that the models that combine different community scores improve predictive performance. Here, we estimate the relative predictive power of different community scores. Table IX shows the performance of the model with one similarity removed. We observe that the topic- and expertise-based metrics are the most useful for our problem. Also, the scores based on language features ($*$-*lang*) and community norms (*c-norm*) are also of modest importance. The availability features ($*$-*avail*) are found to be of low importance.

8.2.5. Pipelining Algorithms. Next, we consider a strategy of combining the ranking algorithms in a pipeline fashion. So to get n recommendations, we run the first algorithm to get $c \cdot n$ recommendations, filter the set of communities, and then run the second algorithm over the filtered set of communities to get n recommendations. The two pipelining strategies are

$$P1 : Borda\ count \rightarrow ranking\ SVM \tag{25}$$

$$P2 : ranking\ SVM \rightarrow Borda\ count. \tag{26}$$

Table X. Performance of the Pipelined Algorithm

	Enterprise			WebMD		
Model	*P*@1	*P*@5	*MRR*	*P*@1	*P*@5	*MRR*
*P*1	**0.67**	**0.86**	**0.76**	**0.55**	**0.88**	**0.71**
*P*2	0.64	0.84	0.73	0.55	0.85	0.68

Table XI. Precision@5 and Time Taken (in Milliseconds) of the *Ranking SVM* Algorithm Using the Partitioned and the Nonpartitioned Strategy

Dataset	Strategy	*P*@5	Time Taken (ms)
Enterprise	*nonpartition*	**0.86**	1,240
	partition	0.79	313
WebMD	*nonpartition*	**0.88**	1,921
	partition	0.80	955

Table X shows the performance of the two pipelining strategies. We set $c = 5$ to run these experiments. We see that pipelining leads to a significant boost ($\sim 3\% - 4\%$) in the performance of the models in comparison to their individual performance (Table IV). This is because the first algorithm filters noisy communities, which leads to a robust ranking by the second algorithm. Similar improvements after removing the noisy items before ranking have also been reported previously [Pal and Counts 2011].

8.3. Community Partitioning

The ranking algorithms evaluated so far compute a match for a routed question q with all the questions, users, and communities in the corpus. It can be quite expensive for a large dataset. In this section, we evaluate a partitioning scheme that clusters communities into l clusters (see Section 6.5). We choose $l = 50$ for the Enterprise dataset and $l = 15$ for the WebMD dataset. Table XI shows the performance of the *ranking SVM* algorithm. For the two methods, we report the time taken to generate the recommendations.

Table XI shows that the partitioning strategy leads to a significant drop in the model performance by 7% to 9%. This makes sense as the partitioning strategy increases the likelihood of excluding the desired community from the target cluster due to clustering criteria. However, it also leads to a reduction in time by 55% to 75%. This is because the target cluster contains fewer communities and hence the algorithm is run on a much smaller dataset.

Note that the time taken does not include the time to parse the community data and compute the entity features like topic distribution and user expertise. These features are precomputed. It should be noted that the overall approach can be parallelized, enabling its integration with a Mapreduce [Dean and Ghemawat 2008] type distributed computing framework, thus making it an attractive choice for real-time systems.

9. DISCUSSION

Our results confirm that the proposed framework yielded more desirable communities than the models that do routing based on top experts or similar questions. Here, we discuss the key aspects of the method we used and attributes of entities that might explain this improvement. First, to what extent do individual features matter when it comes to recommending a set of communities? At the outset of the article, we argued that some combination of topical interest, expertise, and community norms is a likely "sweet spot." For a systematic comparison, we computed the performance of these features in isolation. The results show that apart from topical interest and expertise, language features such as usage of technical terms, personalization, friendliness, and negative sentiments provide a boost of 10% to 14% in the predictive power of the models.

The precise balance between the different features depends on the routed question and the underlying dataset. We saw that the maximum margin approach of *ranking SVM* strikes this balance most effectively and leads to the best generalization accuracy. The primary reason for this improved performance of the *ranking SVM* method is that it directly encodes the pairwise inequality constraints, which lead to a direct optimization of the precision@1. With respect to reducing the running time of the models, we saw that the clustering of communities based on their topical match leads to a reduction of only 7% to 9% in the performance, while leading to a reduction of 55% to 75% in the running time.

The key aspect of our framework is computation of different community scores. The computation of the community scores through *min*-, *max*-, and *mean*-based strategies does not perform well for the routing task. Intuitively, this makes sense because a community consists of different kinds of users and a strategy that overlooks this difference cannot model the communities accurately. *knn* methods register a significant boost in the performance. In particular, *Local-knn* performs better for small k and *Global-knn* performs better for large k. The primary reason for this pattern is the inherent bias in the model, and as a result, we need to pick optimal k based on a holdout dataset. In fact, the scoring models can be sensitive to the input data, and hence, a two-stage pipelining strategy wherein we first filter the input and then run a scoring model can lead to a more reliable score estimation.

In terms of the computation of similarity metrics, we saw that the number of topics and the choice of topic scheme can lead to a drastic difference in the model performance. A naive choice of number of topics could lead to underfitting or overfitting. Hence, selection of the right models is essential for good performance. Lastly but importantly, we saw that ignoring the semantics of users while computing community scores is a less effective strategy, implying that the *knn*-based aggregation strategy is a more attractive choice than a naive aggregation through mixing of the activity.

10. CONCLUSION

In this article, we presented a novel problem of question routing to structured user communities. It is a slightly different problem from routing questions to individuals in online communities. We used several prior proposed measures and added novel measures for modeling different entities of the community ecosystem. We proposed several similarity metrics based on these features and proposed a CUTOFF-AGGREGATION algorithm to get community scores. We performed a comprehensive evaluation of our proposed framework and showed its effectiveness over two large real-world datasets. We also evaluated a community-partitioning technique to speed up the framework and a pipelining strategy to improve the model performance. Our work is a first step to address the community question routing problem, and for the future work, we wish to explore different ranking algorithms and test our recommendation framework on several other datasets.

REFERENCES

Sihem Amer-Yahia, Senjuti Basu Roy, Ashish Chawlat, Gautam Das, and Cong Yu. 2009. Group recommendation: Semantics and efficiency. *Proceedings of the VLDB Endowment* 2, 1 (Aug. 2009), 754–765.

David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2001. Latent Dirichlet allocation. In *Advances in Neural Information Processing Systems 14 Neural Information Processing Systems: Natural and Synthetic (NIPS'01)*. MIT Press, 601–608.

Manuel Blum, Robert W. Floyd, Vaughan R. Pratt, Ronald L. Rivest, and Robert Endre Tarjan. 1972. Linear time bounds for median computations. In *Proceedings of the 4th Annual ACM Symposium on Theory of Computing*. ACM, 119–124.

Mohamed Bouguessa, Benoît Dumoulin, and Shengrui Wang. 2008. Identifying authoritative actors in question-answering forums: The case of Yahoo! answers. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'08)*. ACM, 866–874.

Yunbo Cao, Huizhong Duan, Chin yew Lin, Yong Yu, and Hsiao wuen Hon. 2008. Recommending questions using the mdl-based tree cut model. In *Proceeding of the 17th International Conference on World Wide Web (WWW'08)*. ACM, 81–90.

Shuo Chang and Aditya Pal. 2013. Routing questions for collaborative answering in community question answering. In *Advances in Social Networks Analysis and Mining (ASONAM'13)*. ACM, 494–501.

Kenneth Ward Church. 1988. A stochastic parts program and noun phrase parser for unrestricted text. In *2nd Applied Natural Language Processing Conference (ANLP'88)*. ACL, 136–143.

Don Coppersmith, Lisa Fleischer, and Atri Rudra. 2006. Ordering by weighted number of wins gives a good ranking for weighted tournaments. In *Proceedings of the 17th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA'06)*. ACM, 776–782.

Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: Simplified data processing on large clusters. *Communications of ACM* 51, 1 (2008), 107–113.

Inderjit S. Dhillon, Yuqiang Guan, and Brian Kulis. 2004. Kernel k-means: Spectral clustering and normalized cuts. In *Proceedings of the 10th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'04)*. ACM, 551–556.

Ronald Fagin, Ravi Kumar, and D. Sivakumar. 2003. Comparing top k lists. *SIAM Journal of Discrete Mathematics* 17, 1 (2003), 134–160.

Mike Gartrell, Xinyu Xing, Qin Lv, Aaron Beach, Richard Han, Shivakant Mishra, and Karim Seada. 2010. Enhancing group recommendation by incorporating social relationship interactions. In *Proceedings of the 2010 International ACM SIGGROUP Conference on Supporting Group Work (GROUP'10)*. ACM, 97–106.

Jagadeesh Gorla, Neal Lathia, Stephen Robertson, and Jun Wang. 2013. Probabilistic group recommendation via information matching. In *Proceedings of the 22nd International World Wide Web Conference, (WWW'13)*. 495–504.

Michael Grant and Stephen Boyd. 2008. Graph implementations for nonsmooth convex programs. In *Recent Advances in Learning and Control*, V. Blondel, S. Boyd, and H. Kimura (Eds.). Springer-Verlag Limited, 95–110. http://stanford.edu/ boyd/graph_dcp.html.

Jinwen Guo, Shengliang Xu, Shenghua Bao, and Yong Yu. 2008. Tapping on the potential of q&a community by recommending answer providers. In *Proceedings of the 17th ACM Conference on Information and Knowledge Management (CIKM'08)*. ACM, 921–930.

Ralf Herbrich, Tom Minka, and Thore Graepel. 2007. TrueSkill^{TM}: A Bayesian skill rating system. In *Advances in Neural Information Processing Systems 19 (NIPS'06)*. MIT Press, 569–576.

Liangjie Hong, Ron Bekkerman, Joseph Adler, and Brian D. Davison. 2012. Learning to rank social update streams. In *Proceedings of the 35th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'12)*. ACM, 651–660.

Matthew A. Jaro. 1989. Advances in record-linkage methodology as applied to matching the 1985 Census of Tampa, Florida. *Journal of the American Statistics Association* 84, 406 (1989), 414–420.

Thorsten Joachims. 2002. Optimizing search engines using clickthrough data. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'02)*. ACM, 133–142.

Pawel Jurczyk and Eugene Agichtein. 2007. Discovering authorities in question answer communities by using link analysis. In *Proceedings of the 16th ACM Conference on Information and Knowledge Management*. ACM, 919–922.

Ritwik Kumar, Arunava Banerjee, Baba C. Vemuri, and Hanspeter Pfister. 2011. Maximizing all margins: Pushing face recognition with kernel plurality. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV'11)*. IEEE, 2375–2382.

Liang-Cheng Lai and Hung-Yu Kao. 2012. Question routing by modeling user expertise and activity in cQA services. In *The 26th Annual Conference of the Japanese Society for Artificial Intelligence*.

Baichuan Li, Irwin King, and Michael R. Lyu. 2011. Question routing in community question answering: Putting category in its place. In *Proceedings of the 20th ACM Conference on Information and Knowledge Management (CIKM'11)*. ACM, 2041–2044.

Wei Li, Charles Zhang, and Songlin Hu. 2010. G-Finder: Routing programming questions closer to the experts. In *ACM Sigplan Notices*, Vol. 45. ACM, 62–73.

Jing Liu, Young-In Song, and Chin-Yew Lin. 2011. Competition-based user expertise score estimation. In *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'11)*. ACM, New York, NY, 425–434.

Jing Liu, Quan Wang, Chin-Yew Lin, and Hsiao-Wuen Hon. 2013. Question Difficulty Estimation in Community Question Answering Services. In *EMNLP*. ACL, 85–90.

Qiaoling Liu and Eugene Agichtein. 2011. Modeling answerer behavior in collaborative question answering systems. In *ECIR (Lecture Notes in Computer Science)*, Vol. 6611. Springer, 67–79.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schtze. 2008. *Introduction to Information Retrieval*. Cambridge University Press, New York, NY.

George Lann Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions I. *Mathematical Programming* 14, 1 (1978), 265–294.

Mark O'Connor, Dan Cosley, Joseph A. Konstan, and John Riedl. 2001. PolyLens: A recommender system for groups of user. In *Proceedings of the 7th Conference on European Conference on Computer Supported Cooperative Work (ECSCW'01)*. Kluwer Academic, 199–218.

Aditya Pal and Scott Counts. 2011. Identifying topical authorities in microblogs. In *Proceedings of the 4th International Conference on Web Search and Web Data Mining (WSDM'11)*. ACM, 45–54.

Aditya Pal, F. Maxwell Harper, and Joseph A. Konstan. 2012. Exploring question selection bias to identify experts and potential experts in community question answering. *ACM Transactions on Information Systems* 30, 2 (2012), 10:1–10:28.

Aditya Pal and Joseph A. Konstan. 2010. Expert identification in community question answering: Exploring question selection bias. In *Proceedings of the 19th ACM Conference on Information and Knowledge Management, (CIKM)*. ACM, 1505–1508.

Aditya Pal, Fei Wang, Michelle X. Zhou, Jeffrey Nichols, and Barton A. Smith. 2013. Question routing to user communities. In *Proceedings of the 22nd ACM International Conference on Conference on Information & Knowledge Management (CIKM'13)*. ACM, New York, NY, 2357–2362.

Jenny Preece and Diane Maloney Krichmar. 2005. Online communities: Design, theory and practice. *Journal of Computer Mediated Communication* 10, 4 (2005).

David J. Rogers and Taffee T. Tanimoto. 1960. A computer program for classifying plants. *Science* 132, 3434 (Oct. 1960), 1115–1118.

Lee Sproull and Manuel Arriaga. 2007. Online communities. In *The Handbook of Computer Networks*, H. Bidgoli (Ed.). Wiley Publishing.

Pang-Ning Tan, Michael Steinbach, and Vipin Kumar. 2005. *Introduction to Data Mining*. Addison-Wesley Longman, Boston, MA.

Mao Ye, Xingjie Liu, and Wang-Chien Lee. 2012. Exploring social influence for recommendation: A generative model approach. In *The 35th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. ACM, 671–680.

Dell Zhang and Wee Sun Lee. 2003. Question classification using support vector machines. In *Proceedings of the 26th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'03)*. ACM, 26–32.

Jun Zhang, Mark S. Ackerman, and Lada Adamic. 2007. Expertise networks in online communities: structure and algorithms. In *Proceedings of the 16th International Conference on World Wide Web (WWW'07)*. ACM, 221–230.

Yanhong Zhou, Gao Cong, Bin Cui, Christian S. Jensen, and Junjie Yao. 2009. Routing questions to the right users in online communities. In *Proceedings of the 25th International Conference on Data Engineering (ICDE'09)*. IEEE, 700–711.

Received March 2014; revised January 2015; accepted January 2015